

Design and Analysis of Software Systems – Spring 2025

Assignment 2 – Software Testing (100 marks)

Deadline: 9th March, 8:00 PM

Instructions:

- This assignment is an individual submission.
- All script submissions should be submitted via Moodle.
- Inputs/output should fit the criteria mentioned in respective question.
- All other conditions are open to your interpretations.
- Please ensure that you maintain a clear record of your submission by following the submission criteria with clear code changes and clean organization of files.
- For queries, reach out to TAs via Moodle.

Q1. Helping a friend (30 marks)

Your friend has started a new food company and wants your help to develop an online food delivery system. But before you go into your app developer mode, you want to ensure that you are on the right track. The instructions given by your friend are as follows:

- The app should support both Home Delivery and Takeaway orders.
- The company has its own fleet of delivery agents that will be managed by your system.
- Customers should be able to view the time required/left for their order to be delivered.
- Multiple orders can be placed at once.
- The company manager should be able to view a restaurant POV.
- There should be a persistent instance of the application (for example, if I open 2 instances (terminals) of the app, a single instance should be shared among them).

Tasks

- a. (10 marks) Given that the requirements are from a business POV, write down the software specifications for the system and mention possible use cases. Note that there should be enough major use cases covered (cases like opening and closing of app do not come under major use cases).

- b. (10 marks) Now build a CLI version of the app using Python. The application should be structured into multiple classes with a clear separation of concerns.
- c. (10 marks) Write unit tests using PyUnit to verify all operations and functionalities. There is no limit on the number of test cases, but all possible scenarios should be covered.

Q2. Let's Make a Game! (25 marks)

- a. (15 marks) The task is to make a board game, **Gobblet Jr.**, in Python. You may use Python libraries like [Turtle](#), [Pygame](#), etc. The following are few resources for the game:

- <https://docs.racket-lang.org/games/gobblet.html>
- <https://www.youtube.com/watch?v=nYvYEhBG9HM>

Some clarifications:

- Implement the 3x3 board game having 6 pieces per color (2 of each size).
 - There is no restriction on the design (e.g. you can use circles to represent the pieces or be creative).
 - All the rules of the game as per the document linked need to be implemented.
 - Feel free to make assumptions (as long as all the requirements are fulfilled) and explicitly specify them in the README file.
- b. (10 marks) Now that the game is ready, it is time to make it better. Using Pylint module in Python, conduct static code analysis or lint your program. Following are the deliverables:
 - Submit the initial lint result along with Pylint score.
 - Review the recommendations from your lint results, address them in your code and capture the lint results.
 - Repeat linting by addressing as many recommendations as possible and capture the lint results along with score after every iteration.

Q3. Dollmart (35 marks)

You are hired as a software developer in 'Dollmart', a large-scale supermarket store. They sell everything from groceries to electronics to items of personal well-being. You name it,

they have it. In the world of technology, Dollmart wants you to develop an e-market software for their brand. The deliverables are as follows:

- Customers can be both individuals as well as retail stores (bulk ordering) and pricing will be adjusted accordingly.
- Users should be able to find the item they are looking for intuitively with ease.
- Implement incentives for regular customers by generating discount coupons.
- Delivery of orders needs to be handled properly.

(Since the requirements are not concrete, you are free to make appropriate assumptions and implement use cases accordingly)

Tasks

1. (10 marks) Make a UML class diagram for your e-marketing system.
2. (10 marks) Implement the system (CLI version) in Python in accordance with your UML diagram.
3. (15 marks) Write testcases for your system using PyTest covering all functionalities and all possible edge cases.

NOTE: Document your code property and give appropriate reason behind every choice you make in the development process from division of classes to the softwares used (for database etc).

Resources:

- Pylint - https://pylint.readthedocs.io/en/stable/user_guide/usage/run.html
- Unittest - <https://docs.python.org/3/library/unittest.html>
- PyTest - <https://pytest.org/en/latest/how-to/assert.html>

Submission Format:

Q.1 - Submit in the following format in master folder q1:

- Folder: **src** -> contains your source code for your application.
- Folder: **testcases** -> contains all your testcases associated with your code.
- The SRS should be included in the README itself.

Q.2 - Submit in the following format in master folder q2:

- Folder: **OriginalGame** -> Place your original python script named '**gobblet.py**'. The code can be divided into modules, but your submission must be executable from this file.
- Folder: **InitialLint** -> Place your initial lint result in a text file.
- Folder: **AllLint** -> Include all code versions along with corresponding lint results. Name the final version as '**gobbletfinal.py**' and '**lintfinal.txt**'.

Q.3 - Submit in the following format in master folder q3:

- Folder: **uml** -> Contains your UML class diagram.
- Folder: **src** -> Contains your source code for the application.
- Folder: **testcases** -> contains all your testcases associated with your code.

README.md* (10 marks) - This is a mandatory file, please write notes about all questions (what test cases you have written, why, etc.) and instructions on how to run their unit tests/programs etc. Any other notes to evaluators can be described here. Also add any question-specific requirements mentioned above.

Add all the folders q1, q2, q3 and README.md file in a directory named <rollnumber> and zip the contents as <rollnumber>.zip. If and only if the size of your submission is more than 20 MB, check-in your assignment submission in github and provide us the repository URL in readme.txt. ZIP the readme.txt in <rollnumber>.zip

PS: Use of Code Assist tools and GENERATIVE AI is not permitted.